

Doubly Linked List (Basic Operations)

(Insertion at the end)

```
void insert()
```

```
{  
    Node* // Rest of the code
```

```
    Node* newnode = new Node(val); // creating new  
                                     node
```

```
    if(head == NULL)  
    {
```

```
        head = tail = newnode;
```

```
    }
```

```
    // if 1st node, point both head & tail to newnode
```

```
    else
```

```
    {
```

```
        newnode->prev = tail;
```

```
        tail->next = newnode;
```

```
        tail = newnode;
```

```
    }
```

```
    // if not first then put tail into the  
    prev of newnode.
```

```
    // then put newnode in the next of tail  
    that was the now 2nd last node.
```

```
    // Now point tail to newnode
```

(Insertion at beginning)

```
void insertatb()
```

```
{
```

```
... // Roc
```

```
Node* newnode = new Node(val); // creating new node
```

```
if (head == NULL)
```

```
{
```

```
head = tail = newnode;
```

```
}
```

```
// if 1st node, then point head & tail to newnode
```

```
else
```

```
{
```

```
newnode->next = head;
```

```
head->prev = newnode;
```

```
head = newnode;
```

```
}
```

```
// if not 1st then put head in the next  
of newnode as it is inserted at begin
```

```
// Now put newnode in the prev of head
```

```
// Lastly point head to this newnode
```

(Insert after value)

```
void insertaftval()
```

```
{
```

```
if (head == NULL) { return; }
```

```
// if no node; so return
```



```
Node* temp = head; // to find value
```

```
... // RDC
```

```
while (temp != NULL && temp->data != target)
```

```
{  
    // Run loop until temp becomes NULL or  
    we found target value.
```

```
    temp = temp->next;
```

```
    // traversing temp, incrementing
```

```
}
```

```
if (temp == NULL) { return; } // if val not found
```

```
... // RDC
```

```
Node* newnode = new Node(val); // created node
```

```
newnode->next = t->next; // put next of temp into  
                        next of this newnode
```

```
newnode->prev = temp; // put temp into prev  
                     of newnode
```

```
if (temp->next == NULL) // if next of t != NULL
```

```
{
```

```
    temp->next->prev = newnode; // put newnode  
                                into the prev  
                                of next of temp
```

```
}
```

```
temp->next = newnode;
```

```
// put newnode into next of temp // Dry run clearly  
// explains.
```

```
if (temp == tail) // if temp is at end
```

```
{
```

```
    tail = newnode; // update tail
```

```
}
```

(Void delete by Value)

```
void delV()
```

```
{  
    if (head == NULL) // if no node  
    { return; }
```

```
    // ROC
```

```
    if (head->data == val) // if data is at head  
    {
```

```
        Node* temp = head; // store head in temp  
        head = head->next; // move head to its next
```

```
    }  
    if (head != NULL) // if head != NULL  
    {
```

```
        head->prev = NULL; // move NULL into prev of  
    } // this new head
```

```
    else // if head == NULL  
    {
```

```
        tail = NULL; // point tail to NULL also  
    }
```

```
    delete temp; // delete the node  
    return
```

```
}
```

```
Node* temp = head; // for traversing
```

```
while (temp != NULL && temp->data != val)  
{
```

```
    temp = temp->next;
```

```
} // same as before
```



```
if (temp == NULL) { return; } // if value not found
```

```
if (temp->prev != NULL) // if temp->prev != NULL
```

```
{ temp->prev->next = temp->next; // move next of temp  
// into next of prev of  
temp. prev prev temp next
```

```
if (temp->next != NULL)
```

```
{ temp->next->prev = temp->prev;   
// prev temp next  
this  
prev temp next
```

```
if (temp == tail) // if temp is at end
```

```
{ tail = temp->prev; // move tail to its previous
```

```
delete temp; // delete that node of val
```

(Search by Value)

```
bool sbv()
```

```
{ if (head == NULL) { return; }
```

```
... temp = head;  
Node* new node = new Node(temp);
```

```
while (temp != NULL) // same as SSL
```

```
{ if (temp->data == val)  
{ return true;  
}  
temp = temp->next;  
}
```

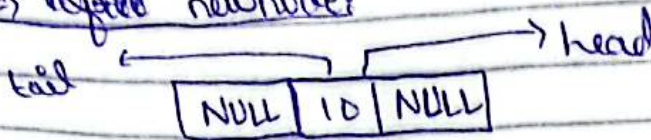
(DRY RUN)

1) Insertion at end():

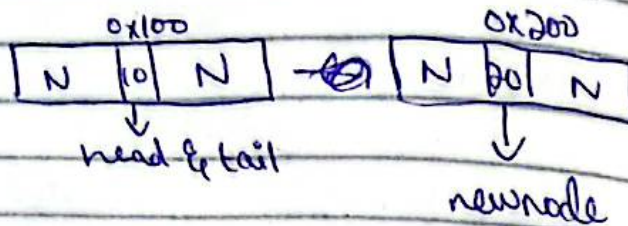
→ First Node:

before → head = tail = NULL;

⇒ After newnode:

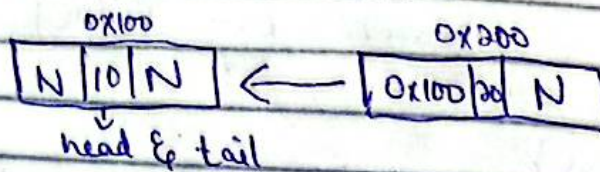


// Now if 2nd node or more nodes insertion,



⇒ Now:

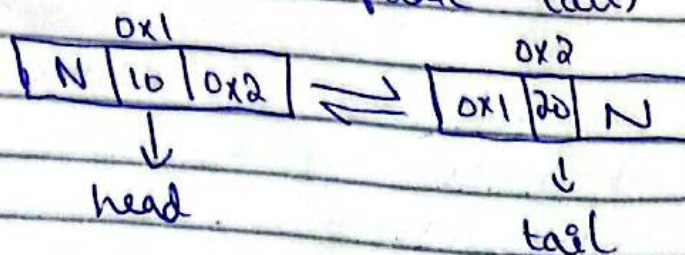
newnode → prev = tail;



tail → next = newnode;



tail = newnode; (update tail)



and So on

Insertional beginning()

⇒ First Node:

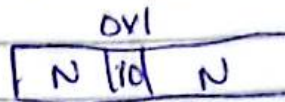
(Same)

⇒ Second or more nodes:



↓

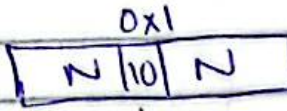
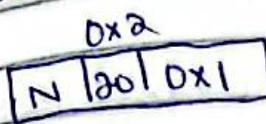
newnode



↓

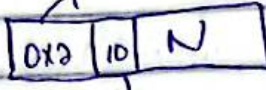
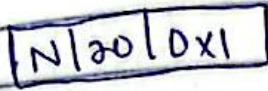
head & tail

newnode → next = head;



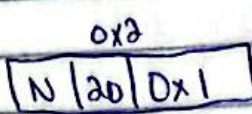
↓
head & tail

head → prev = newnode;



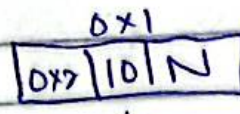
↓
head & tail

head = newnode; (update head)



↓

head



↓

tail

& so on



Insert after value():

Node* temp = head; For traverse

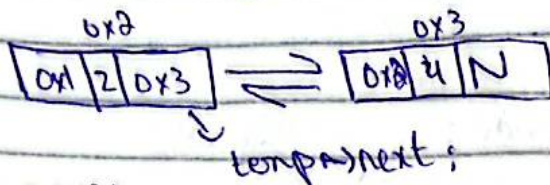
while (temp != NULL) { // to find value after which we have to insert

Suppose:

0x1	N	1	0x3
0x2	0x1	2	0x3
0x3	0x2	4	N

→ temp is here, we will insert after this:

0x4
[N | 3 | N] → newnode created



⇒ now:

newnode → next = temp → next;

[N | 3 | 0x3]

newnode → prev = temp;

[0x2 | 3 | 0x3]

→ Now:

if (temp → next != NULL)

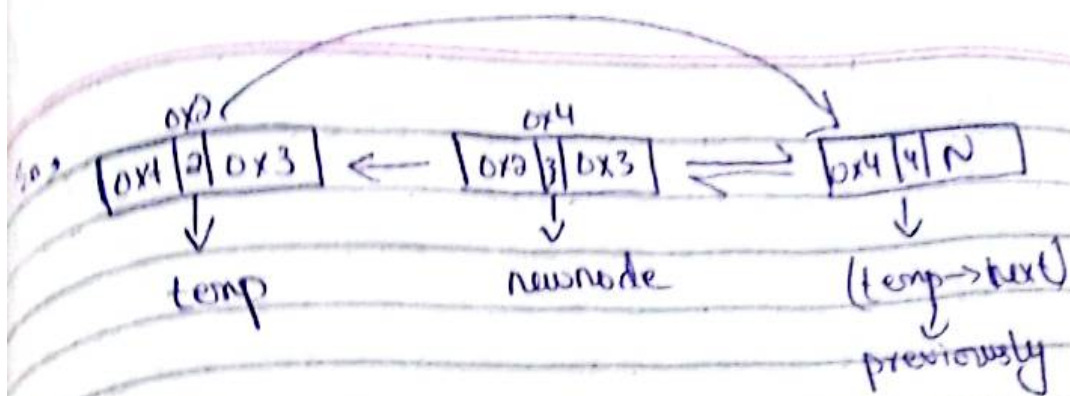
{ temp → next → prev = newnode;

[0x1 | 2 | 0x3]

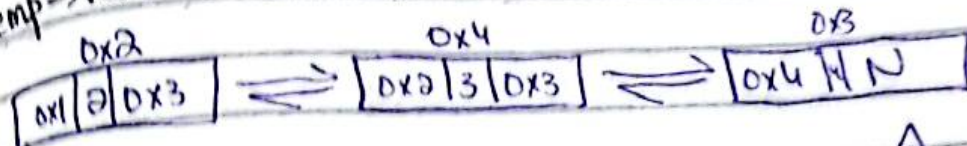
temp temp → next

[0x2 | 4 | N]

temp → next



temp -> next = newnode;



if (temp == tail)

{ tail = newnode; // simply update tail if this "was" NULL. }

1) Delete by value

// if head is about to del just skip head to its next & move NULL into the prev of new head if it is not NULL if NULL then point tail to NULL also.

Node* temp = head; // For traversing

while (temp != NULL) { // until we found val to be del, move temp to its next for every iteration. }

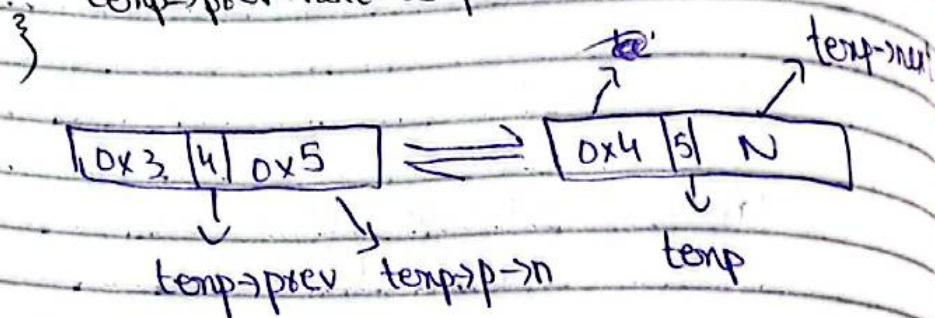
if (temp == NULL) { return; } // if val not found

0x1	0x1	1	0x2
0x2	0x2	2	0x3
0x3	0x3	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	N

→ Suppose temp is here now

```
if (temp->prev != NULL)
{
```

```
temp->prev->next = temp->next;
```



So,



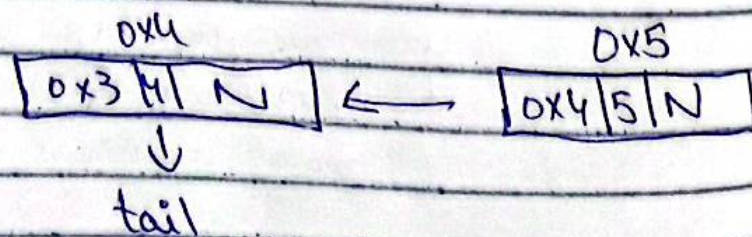
```
if (temp->next != NULL)
{
```

```
    } // as it is NULL
```

```
if (temp == tail)
{
```

```
    tail = temp->prev;
```

So,



delete temp now.

update singly linked list:

Node* temp = head

Simple traversal, if val found
prompt it to return true otherwise false

void Update()

Node* temp = head;

// for traversal

Example 8

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	N

let's update (0x3) (3) to (5)

let temp = 0x1;

temp != NULL
temp->d = val X
temp = temp->n;

if temp == 0x2;

temp != NULL V
temp->d = val X
temp = temp->n

if temp == 0x3

temp != NULL V
temp->d = val V
{
temp->d = 5;}

So,

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	5	0x4
0x4	0x3	4	N



our new list

void display();

Same as
SLL

Doubly Linked List (All Questions)

(Question # 01)

void sortedinsert()

if(head == NULL)

{ head = tail = newnode; }

// if no node before

if(val <= h->data)

// if val is < head
// insert at beginning

ROC;

if(val >= tail->d)

// if val > tail
// insert at end

ROC;

Node *temp = head;

// for traversal

while(temp != NULL && temp->data < val) // if, then run

temp = temp->next;

// iterate temp

newnode->next = temp;

newnode->prev = temp->prev;

temp->prev->next = newnode;

temp->prev = newnode;

managing nodes
Linking
Before

(DRY RUN)

Suppose: Our list is
to be
value entered is 2

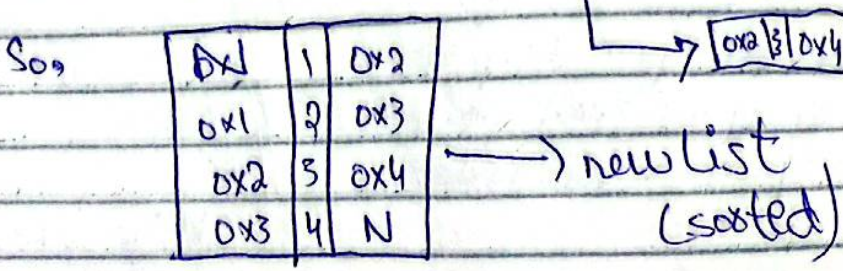
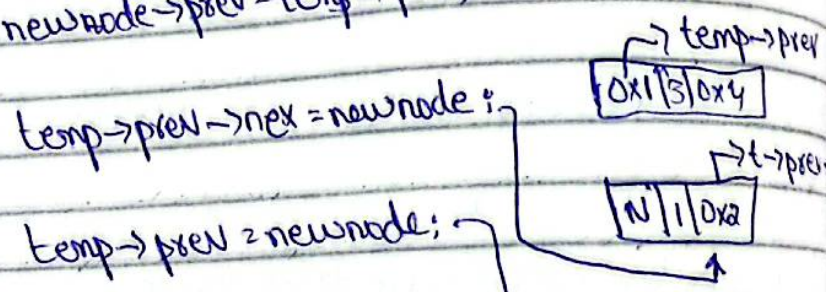
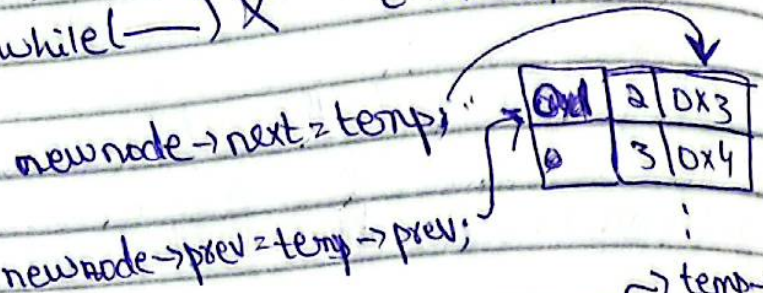


-) if (head == NULL) X
-) if (val < head->data) X $2 < 1$ X
-) if (val > tail->data) X $2 > 4$ X

temp = 0x1 (1)

-) while (t != N && t->data < val) ✓
 - {
 - t = t->next;
 - t = 0x3
 - }

-) while (—) X t->data < val



Question # 02

void deleteOccurrence()

{ if (head == NULL) { return; }

// list empty

Node* temp = head;

// for traversal

int count = 0;

// for counting occurrences

while (temp != NULL)

{ if (temp->data == val)

// if val found

{ Node* todelete = temp;

if (temp == head)

// if at head

{

Managing head;

}

else if (temp == tail)

// if at tail

{

Managing tail;

}

else

// if b/w somewhere

{

temp->prev->next = temp->next;

temp->next->prev = temp->prev;

temp = temp->next;

// ~~Deleting~~
Linking prev &
next of temp

}

delete todel;

count++;

// increment c

}

else { temp = temp->next; }

// increment temp

}

DRY RUN

Suppose: Our list is
values to del 2
temp = 0x1 (1) (head)

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	2	0x4
0x4	0x3	3	0x5
0x5	0x4	3	N

```

1) while (t != NULL) ✓
{
    if (t->data == zval) X
    else
    {
        t = t->next;
    }
}

```

t = 0x2

```

2) while (t != N) ✓
{
    if (t->d == val) ✓
    {
        Node* todel = t;
    }
}

```

todel = 0x2

```

if (t == head) X
else if (t == tail) X
else
{

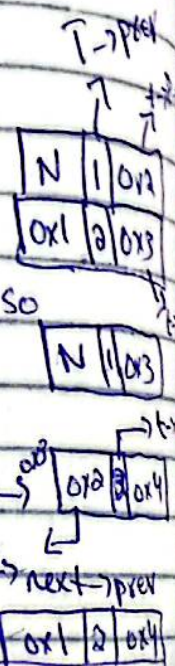
```

```

    t->prev->next = t->next;
    t->next->prev = t->prev;
}

```

t = t->next; (t = 0x3)



delete t->next;

c++;

again while(1) ✓

if(1) ✓

t->next = t;

t->next = 0x3

if(t == h) ✗

else if(t == tail) ✗

else

{

t->next->next = t->next; →

t->next->prev = t->prev;

t = t->next; (t = 0x4)

}

delete t->next;

c++;

c=2

again: while(1) ✓

if(1) ✗

else

t = t->next;

t = 0x5

again while(1) ✓

if(1) ✗

else

t = t->next;

t = N

again

while(1) ✗

t = N

0x1	0x1	1	0x3
0x3	0x1	2	0x4
0x4	0x3	3	0x5
0x5	0x4	3	N

Before

0x1	1	0x3
0x3	2	0x4
0x4	3	0x5

After

N	1	0x4
0x1	3	0x5

0x1	N	1	0x4
0x4	0x1	3	0x5
0x5	0x4	3	N

Final List

Question # 03

```
void sum()  
{  
    if (—) { return; } // 0 or 1 node
```

```
    Node* left = head;  
    Node* right = tail;
```

```
    int count = 0;
```

```
    while (L != R && L->prev != R)
```

```
    {  
        int CS = L->data + R->data;
```

```
        if (CS == S)
```

```
        {  
            (prompt) count++;
```

```
            left = L->next;
```

```
            R = R->prev;
```

```
        }
```

```
        else if (CS < S)
```

```
        {
```

```
            L = L->next;
```

```
        }
```

```
        else
```

```
        {
```

```
            R = R->prev;
```

```
        }
```

DRY RUN

Suppose: value to find sum = 17
be ~~deleted~~

Left = head
 Right = tail
 int count = 0
 L = 0x1
 R = 0x7
 C = 0

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	0x6
0x6	0x5	6	0x7
0x7	0x6	7	N

while () ✓
 int CS = L → d + R → d; (CS = 7 + 8)

if (CS = 8) X 8 ≠ 7
 else if (CS < 8) X 8 < 7
 else
 { R = R → prev; } R = 0x6

again: while () ✓
 int CS = L → d + R → d; (CS = 8 + 7)

if (CS = 8) ✓ 7 = 7

prompt;
 count++;
 L = L → next; L = 0x2
 R = R → prev; R = 0x5

again: while () ✓
 in CS = L → d + R → d; (2 + 5 = 7)

if () ✓
 { prompt;
 count++;
 L = L → next; L = 0x3
 R = R → prev; R = 0x4

again: while(—) ✓

ind $CS = L \rightarrow R \rightarrow d;$

if(—) ✓

Prompt

ch;

$L = L \rightarrow next;$

$R = R \rightarrow prev;$

3 ...

0x4
0x3

again: while(—) ✗ (left $\rightarrow$ prev $\neq$ right)

So, pairs will be: (1, 6), (2, 5), (3, 4)

Question # 04

```
void rotate()
```

```
{
```

```
if(—) {return}
```

// 0 or 1 node

```
length = getLength();
```

// found total nodes

```
n = n % length;
```

// to get n in range

```
if(n > 0)
```

// no need to rotate

```
if(n < 0)
```

// opposite rotate

-5 $\rightarrow$ 0

-4 $\rightarrow$ 1

-3 $\rightarrow$ 2

-2 $\rightarrow$ 3

-1 $\rightarrow$ 4

RDC

Detail in (DAY RUN)

DRY — RUN

Suppose $n=2$, our list is :

length = 5
temp = 0x1
count = 1

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	N

while(count < n && t != NULL) ✓

{ t = t->next;
count++;

t = 0x2

c = 2

} while()

2 < 2 ✗

if(t = NU || t->next = NULL) ✗

Node* newhead = t->next

newhead = 0x3

tail->next = head;
head->prev = tail;

temp->next = NULL;
newhead->prev = NULL;

0x5	1	
	2	NULL
NULL	3	
	4	
	5	0x1

temp
newhead

head = newhead;
tail = temp;

}

	3	
	4	
	5	
	1	
	2	NULL

head

tail

Question # 05

```
void merge(h1, h2)
```

```
{  
    Node* h1 = head1;
```

```
    Node* h2 = head2;
```

// storing both

```
    if (h1 == NULL)
```

```
    {  
        head = h2;  
        update tail;  
        return;
```

// if h1 is

```
    }  
    if (h2 == NULL)
```

// if h2 is empty

```
    {  
        return;
```

```
    }  
    Node* new head = NULL;  
    Node* new tail = NULL;
```

// for merged

```
    if (h1->data < h2->data)
```

```
    {  
        new head = h1;  
        h1 = h1->next;
```

// if yes, store h1 & iterate

```
    }  
    else
```

```
    {  
        new head = h2;  
        h2 = h2->next;
```

// if yes, store h2 & iterate

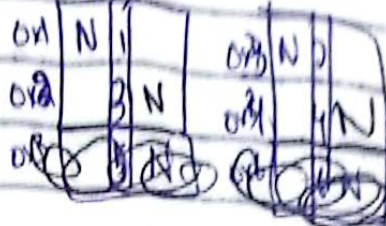
```
    }
```

```
    new tail = new head; // same;
```

```
    new head->prev = NULL; // update prev
```

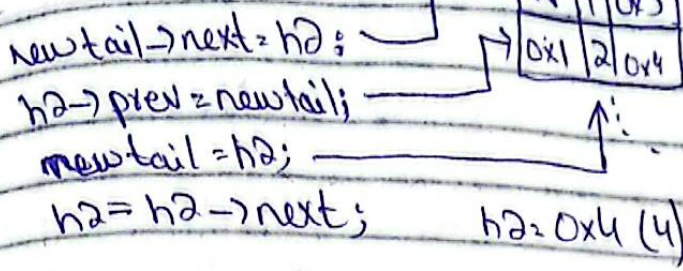
RDC explained in DRY - RUN

Suppose our lists are



prev code same as
SLL merging:

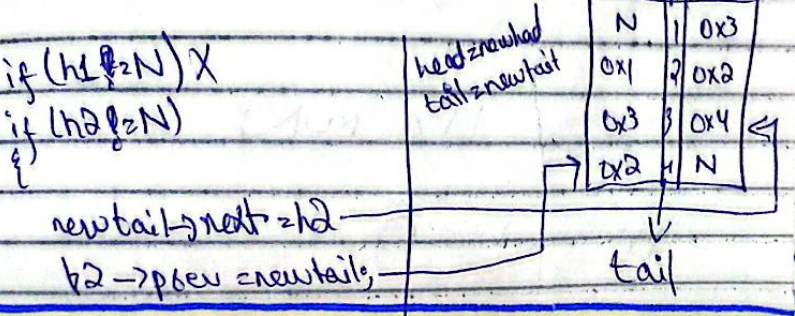
```
while (1) {
    if (h1->d <= h2->d) {
        newtail->next = h1;
    } else {
        newtail->next = h2;
    }
}
```



```
again while (1) {
    if (h1->d <= h2->d) {
        newtail->next = h1;
        h1->prev = newtail;
        newtail = h1;
        h1 = h1->next;
    }
}
```



```
again while (1) {
    if (h1 == N) {
        head = newtail;
        tail = newtail;
    }
}
```



Question # 06

```
void displaytree()
// later in tree
```

```
void inorder()
// later in tree
same just prompt replaced by
insertion
```

```
void convert()
// calling inorder
```

Question # 07

```
if(headz —) // 0, 1, or 2 nodes
```

```
bool found = false;
```

```
Node* first = head;
```

```
while(—)
```

```
{
```

explained in (DRY_RUN)

```
}
```

```
if(found == false) // not triplets
{ return; }
```

DRY RUN

Suppose: $n = 5$

Our list is
 $first = head (0x1)$

$\text{while}(f \neq N) \checkmark$

Node* $S = F \rightarrow next;$ $S = 0x2$
 Node* $T = tail;$ $T = 0x4$

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	0x6

$\text{while}(\rightarrow) \checkmark$

int $sum = F \rightarrow d + S \rightarrow d + T \rightarrow d;$ $(1 + 2 + 4 = 7)$

if $(sum \neq n) \times$ $8 \neq 5$

else if $(sum < n) \times$ $8 < 5$

else

$\{$
 $third = T \rightarrow prev;$ $T = 0x3$

again $\text{while}(\rightarrow) \checkmark$

int $sum = F \rightarrow d + 3 \rightarrow d + T \rightarrow d;$ $(1 + 2 + 3 = 6)$

if $(sum == n) \checkmark$

$\{$
 prompt, true!

$S = S \rightarrow next;$ $S = 0x3$

$T = T \rightarrow prev;$ $T = 0x2$

again $\text{while}(\rightarrow) \times$ $(T \rightarrow next = S)$

! and so on
 again after iterating first
 inner loop

Question # 08

```
void reverse()
```

```
{ Same as of SLL;
```

```
}
```

```
void reversehelper(Node* head, int k)
```

```
{ // Explained in DRY_RUN, remaining same as SLL just new is prev.
```

(DRY_RUN)

Suppose: Our list is :

k=2

0x1	N	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	NULL

prevnode = NULL;

curr = 0x1

NN = NULL;

C = 0

while() ✓

NN = curr->next;

curr->next = prevnode; →

if (NN != N) X

prevnode = curr

curr = NN;

C++;

NN = 0x1

curr = 0x2

C = 1

if (NN != NULL) {

head->next = reversehelper(curr->next, k);

// This will be called recursively



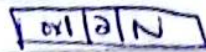
Node PN = Null;
Curr = head
NN = Null;
int c = 0;

→ It is O(2n)

while(→) ✓

{ NN = C → Next ;
C → next = PN ;

NN → 0x3



if (PN != N) X
prevN = curr
curr = NN;

PN = 0x2

curr = 0x3

if (NN != NULL)

h → next = recu(→);



and so on

remaining part is just same as
SLL reversing in groups.

just a new check on prev

Question # 10

```
void split(CLL &sh, CLL &shh)  
{  
    if (head == NULL) // if no node
```

```
        Node *slow = head;  
        Node *fast = head;
```

```
    while (fast != NULL)  
    {  
        fast = fast->next; // same, finding mid
```

```
    }  
    Node* h1 = head;  
    Node* h2 = slow->next; // separating head
```

```
    tail = head->prev; // for saving prev
```

```
    slow->next = h1; // separating first half
```

```
    h1->prev = slow; // making first half a CLL
```

```
    tail->next = h2; // updating h2's prev
```

```
    h2->prev = tail; // updating h2's prev
```

```
    sh.head = h1; // Assigning separate heads
```

```
    shh.head = h2;
```

(DRV_RUN)

Suppose our list is

slow = 0x3
fast = 0x5

h1 = 0x1
h2 = 0x4

tail = 0x6

0x1	0x6	1	0x2
0x2	0x1	2	0x3
0x3	0x2	3	0x4
0x4	0x3	4	0x5
0x5	0x4	5	0x6
0x6	0x5	6	0x1

slow → next = h1

0x2 → next = 0x1

h1 → prev = slow;

0x2	2	0x1
-----	---	-----

0x3	1	0x2
0x4	2	0x3
0x2	3	0x1

tail → next = h2;

h2 → prev = ^{tail}tail;

0x5	6	0x4
-----	---	-----

0x6	4	0x5
0x4	5	0x6
0x5	6	0x4

fh.head = h1;

sh.head = h2;

// Two list created by separated